

1. **Use Appropriate Indexes:** Create indexes on columns frequently used in `WHERE`, `JOIN`, and `ORDER BY` clauses to speed up data retrieval. Avoid over-indexing to prevent slowing down inserts/updates
2. ****Avoid SELECT ***:** Retrieve only the columns you need instead of using `SELECT *`, reducing data transfer and memory usage
- 3.
4. **Use EXPLAIN to Analyze Queries:** Run `EXPLAIN` (or `EXPLAIN ANALYZE` in MySQL 8.0+) to understand the query execution plan and identify bottlenecks
5. **Limit Result Set:** Use `LIMIT` or pagination to restrict the number of rows returned, especially during debugging or when full dataset isn't needed.
6. **Optimize WHERE Clauses:** Apply filters that reduce the result set as early as possible; avoid functions on indexed columns in `WHERE` clauses because these prevent index usage
7. **Rewrite Subqueries as JOINS:** Use `JOIN` instead of subqueries or `IN` clauses when possible to improve readability and performance
8. **Replace OR with IN or UNION:** Using `IN` or `UNION` can be more efficient than multiple `OR` conditions in `WHERE` clauses for better index utilization
9. **Use Suitable Join Types:** Use appropriate joins (`INNER JOIN`, `LEFT JOIN` etc.) based on the data needs to reduce unnecessary row combination
10. **Create Composite Indexes:** For queries filtering by multiple columns, composite indexes on those columns improve performance more than multiple single-column indexes
11. **Partition Large Tables:** Partition tables by key ranges (e.g., date) to limit scanned data and improve query speed on big datasets.
12. **Avoid Wildcard Characters and Functions in Filters:** Such as avoiding leading `%` in `LIKE` or using functions on columns in `WHERE` that block index usage.
13. **Use Covering Indexes:** Design indexes that include all columns needed by the query to avoid accessing the table at all.

14. **Use Consistent Data Types:** Ensure columns used in joins and filters have matching and appropriate data types to avoid implicit conversions and slowdowns.
15. **Cache Frequent Results:** Use application-level caching or query result caching in MySQL (if applicable) to reduce repeated query processing.
16. **Analyze and Profile Regularly:** Monitor slow queries using tools like `SHOW PROFILE` or MySQL Workbench profiling, then tune accordingly.

Where

Examples

```
select first_name from actor
where actor_id IN (
    Select actor_id from
    film_actor where film_id = 5);
```

```
select a.first_name from actor a
inner join film_actor fa on a.actor_id = fa.actor_id
where fa.film_id = 5;
```

```
select * from customer where last_name = 'Ali' Or last_name = 'Jones';
```

```
Select * from customer where last_name IN ('Ali' , 'Jones');
```